

Unit 1 **SEARCH**

Context:

An **intelligent agent** – this is an entity (could be a robot, could be an automated system or a component of an automated system, could be a program that implements some A.I. algorithm) that:

- **Senses** the environment – either by using its own sensors, or by being provided with information about the environment as part of a set of input data provided to the agent and updated as needed.

- **Takes action** in order to achieve a predefined **goal** - Action here doesn't necessarily mean a physical action, it more generally indicates the agent in some way affect the state of the environment, for example, by changing the value of parameters that control what the agent is doing, or by making a choice between possible actions it could take in the future, or by making decisions regarding what is happening in the environment.

- The **goal** - defines what the agent is intended to achieve, and is usually encoded in the form of a **utility function** that the agent has to maximize (alternately, and often seen in machine learning, the agent has a **loss function** it's trying to minimize).

(Very simplified) **Examples of agents:**

- A **smart** thermostat: goal is to keep the temperature inside a room at a comfortable level
 - senses the current room temperature
 - actions include turning the heating or A/C on/off as needed
- A car's simplest **autopilot**: goal is to keep the car driving at a constant speed within its lane
 - senses the car's velocity and location w.r.t. the lane and other cars
 - actions include accelerating/braking, and steering as needed
- A smartphone's **assistant**: goal is to provide a suitable response to user queries
 - senses the user's voice to detect requests for assistance
 - actions include carrying out on-line searching, performing tasks like scheduling, saving common queries, providing voice feedback

(Very simplified) **Examples of non-agents:**

- A light switch: does not have a goal, it simply provides a way for a user to toggle lights on/off
- A traffic light: produces a time-dependent, pre-programmed pattern, does not respond to the environment (caveat: there are smarter versions, which may qualify as agents)

We will look at many types of agents in this course, so the above description is necessarily very general. Details will fill-in as we look into specific A.I. problems and techniques for solving them.

Agents (continued).

Simplest type of intelligent agents: Reactive agents (like the thermostat example above). They sense the environment and their action is decided entirely as a response to the current conditions in the environment.

Control systems (e.g. for industrial processes, vehicles, and so on) are a good example of this kind of agent. They are incredibly useful for automating things, but not very smart.

More interesting: Agents that rely on **a model** of the environment to **make a prediction** regarding what will happen in the near future, and what the effect of the agent's actions will be, and make decisions/take actions accordingly. These are the type of agents we usually think of when we speak of **intelligent agents**.



Search Problems

The first general technique we will discuss for solving problems is **search**. This is a completely general technique that can solve any problem you have (if you are willing to wait long enough). How? it's very simple: Given a problem, and a concrete definition of how a solution to this problem is specified, *try out every possible solution until we find the best (or the correct) one*.

Of course that turns out to be not a very practical way of solving any real problems, so we have to be a bit more concrete regarding what problems we can solve realistically using search, how we specify these problems, and how we go about finding a solution.

Components of Search Problems:

- A definition of what a **configuration or state** for the problem we are solving looks like.
In order to solve a problem using search, we need to determine what are the relevant variables that define the state of the environment – any variables or quantities whose values affect the outcome of our problem need to be included, we call these variables **state variables**. A particular assignment of values to these variables is a **state** or **configuration** for the problem.
- A definition of how **actions** taken by our **agent**, or by any other **agents** that are involved in this problem can change the state of the problem. We need to know how actions affect the values of variables, which allows us to determine given the **current configuration** what are possible **future configurations** depending on what actions are taken by the agents involved in the problem.

This can be **deterministic** or **stochastic**. The information we need can be represented by a **Transition Function** $T(s,a,s')$ which given the current state s , and the action a performed by an agent, gives the **probability** that the next state will be s' . If the problem is **deterministic**, then we can simply think that $T(s,a)$ returns the next state s' . But for most of the interesting problems we can find out there, transitions are **stochastic**, and so we have to account

for multiple possible successor states.

We will encounter the transition function $T(s,a,s')$ soon, as a component of classic **reinforcement learning** algorithms.

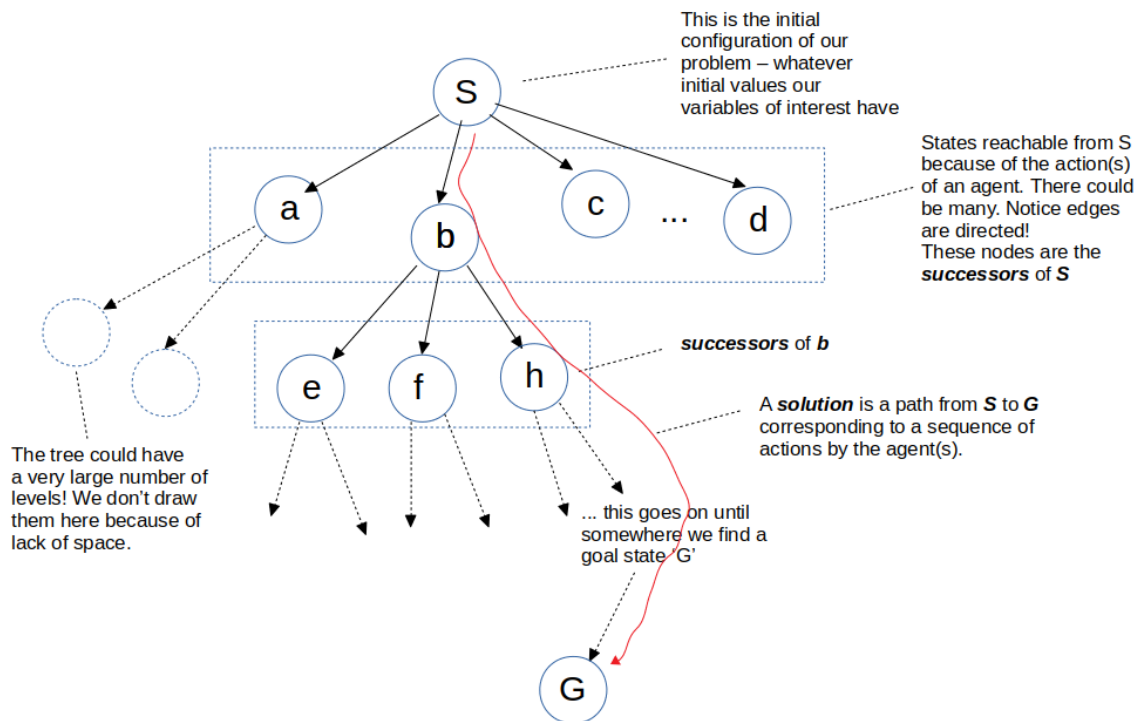
- A definition of a **goal state**. This is simply the desired outcome for the problem we're solving, characterized in a way that can be checked by looking at the values of variables in our **state**. There can be **multiple goal states** for any given problem, and they can have different value (i.e. some may be better than others).

The natural representation for a search problem is a **graph**. The graph contains a set of **vertices** (*also called nodes*) that represent the different **possible configurations** for the problem, and a set of **edges** that connect nodes that are reachable from one another as a result of agent actions.

In typical search problems, the graph takes the form of a **tree** (there is no reason to evaluate a given configuration more than once – if it is the goal, we're done, if it's not, then we move along – there are no loops in the graph because of this).

A **solution** is a sequence of **actions** that leads from the **initial state** to **(the/a) goal state**. It is therefore also a **traversal in the tree** that represents our problem.

The setup of a general search problem is shown below:



Search Tree:

- Contains a **root** node corresponding to the **initial configuration** for our problem
- Nodes correspond to **states (configurations)**
- Each level contains **states** reachable from nodes in the **previous level** that result from possible actions the agents can take
- A **path** in this search tree corresponds to a **plan**. A sequence of actions that takes the agent from the initial state to some other state in the tree. If the final node is a **goal node**, the **plan** is a **solution** for the problem
- For most of the actually interesting problems we may care to solve, the search tree is **too large to explore exhaustively**

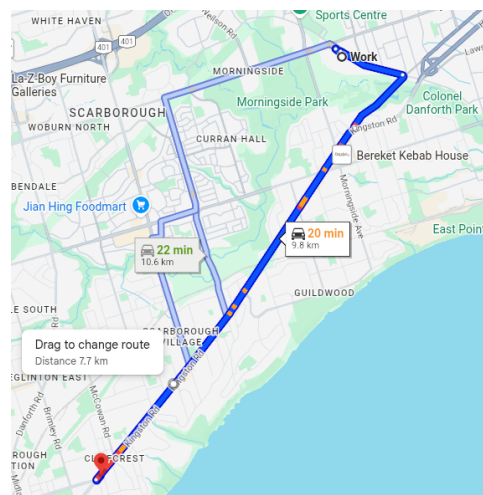
Important: The **search tree** that is explored during the search process **is not the graph encoding the search problem!** The graph encoding the problem is unique, can and does contain loops, and does not provide a sequence of steps to get to the solution, the **search tree** is a realization of a search process, is **not unique** (different search strategies, or even the same strategy with different parameters will produce a different tree), and most often involves only **part of the problem graph**.

Concrete Example

To make things clear, let's have a look at how a specific problem gets translated into a search problem we can handle using the ideas set above.

Problem: **Finding a path between two specific locations**

Example situations where this comes up: **Maps! (Waze, Google maps), Network routing.**



Definition of states (configurations) for the problem: **Possible locations the agent can be at.**
in the maps example, we can divide our map into **a grid**, **each grid location** is a

possible **state**. This is simple, but not optimal – think about **alternate ways** to representation possible locations in this specific case, and consider **limitations** and **advantages** of different representations.

When deciding on your **state definition** and the **state variables** required to specify it, you want to have carefully thought about the effect of your choice on the search process.

Successor states: Locations that can be reached from the current one by driving (safely and legally!). This depends in some way on **how you encode location**. And must also account for the actions available to the agent – directions will vary for **walking** vs. **driving**, vs. **Biking** because the actions available are different and thus the **successor states** change.

Initial state: Starting point for our path finding problem

Goal state: Destination location (or locations, if more than one exists)

Solving Search Problems

The process for solving a search problem is trivial (and you covered it in A48!) – it is simply a graph search that starts at the initial node, and proceeds in some order to explore nodes that can be reached by different sequences of actions, until a goal node is found. The general algorithm is shown below in pseudo-code:

```
Initialize starting node (i.e. determine the root of the search tree)
Create an initially empty list of expanded nodes
Add starting node to the list

while the list is not empty:

    get the next node(*) from the list [implicit: remove it from the list]
    if this node is a goal node
        success! : return path from start node to this goal node
    else
        add successor nodes for this node to the list in some order(**)
        keeping track of the predecessor to each of the nodes just added
        (so the path can be re-constructed when the goal is found)

list is empty and didn't find a goal : failure! (return an empty path)
```

(*) The order in which we **get the next node** is important – see below

(**) The order in which we **add successor nodes to the list** is also important – see below

As you see, the process is quite simple – there's some book-keeping involved: namely, for each node expanded, we keep track of the predecessor – and for many applications, also the **length or cost** of the path. But the algorithm itself is simple. The interesting bits are the details of specifying **in which order** we get the next node from the list, and **in what order** we add successor nodes for a given state to the list. The overall goal of **smart search** is to **find a good path** (under some definition of **good**) with **the least amount of search possible**. This is not trivial for realistic problems, and it is critical for

handling large search problems such as providing driving directions in a large city with hundreds of thousands of possible locations.

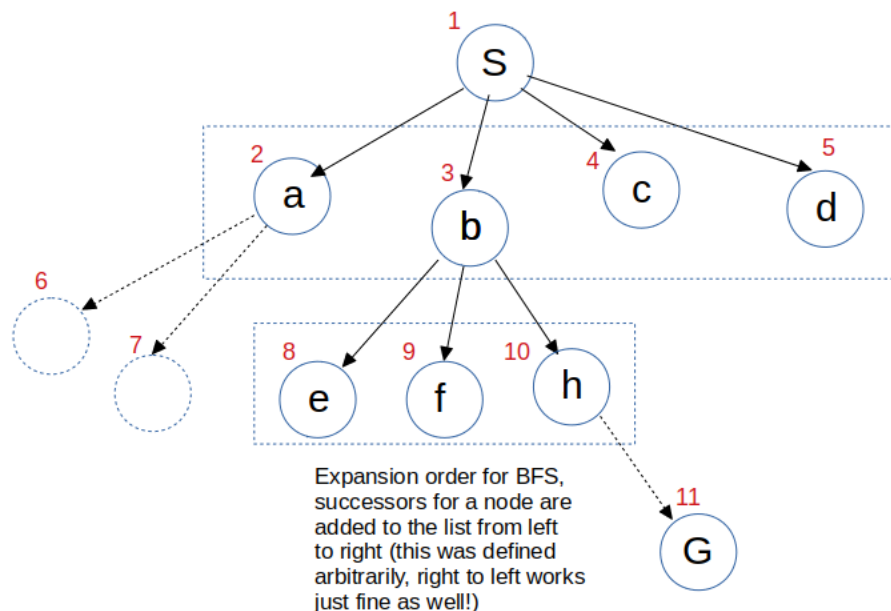
Search ordering

The basic search algorithms you are already familiar with can be employed to solve the search problem described above. These are Breadth First Search (BFS) and Depth First Search (DFS).

BFS adds successor nodes to **a queue** – each node added goes to the end of the queue. The **get the next node from the list** operation then gets the node at the **front** of the queue. So nodes are expanded (explored, checked to see if they are the goal node) in the order in which they arrived at the queue.

This produces a search ordering in which all nodes at level 1 (just below root) are expanded before any nodes at level 2 (two down from root). All nodes at level 2 are expanded before any node at level 3 is expanded, and so on. The search explores the tree level by level, and within a given level the search ordering is either arbitrary (doesn't matter as long as there is a consistent rule to add successor nodes to the list), or can be dependent on the problem we're solving.

BFS is easily implemented with the pseudocode above by using a **queue** to store the list of nodes.

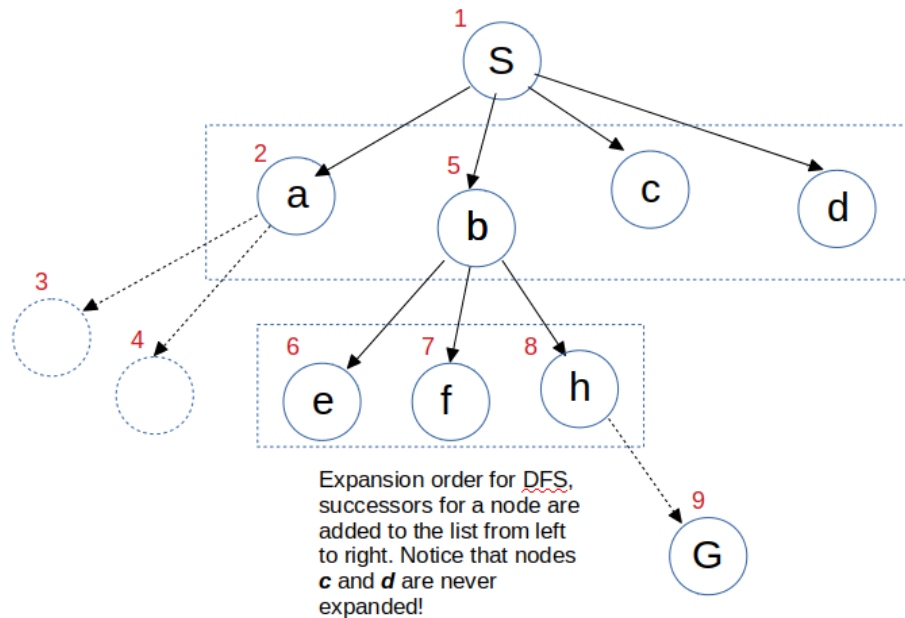


You should replicate the BFS process on paper, and see how the queue works to produce the search ordering shown above.

In contrast to BFS, Depth First Search (DFS) expands nodes in the order that leads to travelling quickly away from the starting node. This is achieved by using a **stack** to keep the list of nodes.

Successor nodes are added to this stack in some order, and the *get the next node from the list* operation gets whatever node is at the top of the stack at any given point.

DFS is more naturally implemented using a recursive function, and if you do this carefully, the recursion keeps track of the path from the start node to any goal discovered during search.



You should verify on paper that the stack produces the search ordering shown above. Both BFS and DFS should be very familiar to you, and you should be able to implement them without much difficulty.

Generalizing Search

The naive search formulation described above needs to be extended a bit in order to handle more interesting problems.

First off – different actions are typically associated with different **cost**. The idea here is that from the point of view of our agent, or in the context of the problem we’re trying to solve, some actions are considered better than others.

For example, suppose there are two roads that have the same length, and lead to the same location. However, one is completely congested with traffic while the other is free. Both of these roads can be part of a plan that reaches the same destination, but one is going to be much worse than the other for the agent that chooses it.

In general, between multiple choices the agent can make at any given time, some will turn out better or result in a solution that is preferable for the agent in some concrete way.

In order to encode the cost of different actions, we allow edges in our search tree to have different weights. The weight of the edge leading from some node A to some node B represents the cost of the action that changes the state from A to B.

Given a graph with weighted edges, ***we no longer want just any solution*** to the search problem. We are interested in finding ***an optimal solution***. This normally means the solution with the ***lowest cost*** for the agent (e.g. in the maps example, the path between two locations that takes us there in the smallest amount of time). Note that there may be multiple solutions with the same optimal cost, we don't have a preference between them, any of those will do!

How do we search for an optimal path?

Uniform Cost Search (UCS)

UCS is a generalization of BFS that works as follows:

- * Use a ***priority queue*** to keep the list of nodes to expand. Nodes in this queue are ***ordered by increasing cost*** to get there from root
- * When we expand a node ***i*** that is not the goal node:
 - Add each neighbour ***j*** of node ***i*** to the priority queue with a priority equal to the ***total cost*** required to get to ***j*** from the ***start node***
 - If a neighbour ***j*** is ***already*** in the priority queue, check if it can be reached with a lower cost by going through ***i***. If so, update its cost in the priority queue
- * The ***get next node to expand from the list*** operation gets the node from the priority queue with the lowest cost to reach from the root

UCS will find the ***shortest path***, or equivalently, the ***lowest cost path***, ***optimal path*** between the start node and a goal node. If all edge weights are set to the same constant value, it is equivalent to BFS.

You may already be familiar with Dijkstra's Shortest Path algorithm, UCS is similar and can be implemented in much the same way – the main differences being that UCS works on a search tree, and stops as soon as a goal node has been discovered, whereas Dijkstra's method will expand the entire graph and find shortest paths to every reachable node.

Importantly, ***UCS is guaranteed to find the optimal path to a goal node*** if any such path exists under the condition that all edge weights are positive and greater than zero.

Why?

- * The search begins with the ***initial node*** at a cost of 0. This is correct by definition.
- * From this point on, the ***priority queue*** ensures that the ***next node to be expanded*** has the ***next smallest cost*** to be reached among all the ***unexpanded nodes***
- * A proof of correctness (by induction) is very similar to the proof of correctness for Dijkstra's

algorithm, except for the stopping condition

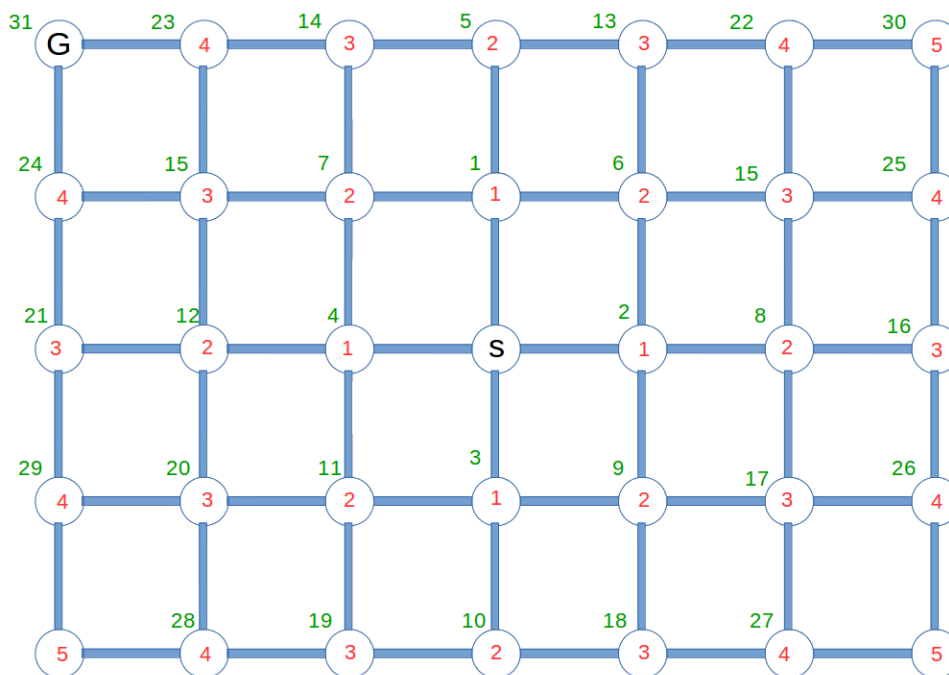
Heuristic Search

There's nothing new or **intelligent** about the search process described above. It is **brute force**. However, such an approach may eventually become unwieldy or impractical.

The computational complexity of BFS, DFS, and UCS on a graph with N nodes and M edges is $O(N+M)$ since it must expand every node and explore every edge in the worst case. Consider how this plays out in an application (such as driving directions) where the search has to be **updated in real time** to account for **changing conditions** such as traffic, accidents or hazards along the route, etc.

This motivates the need to find an algorithm that finds the **optimal path to the goal** while reducing the amount of search required (in terms of the number of nodes expanded and edges examined). The key idea is that at many points during the search, there will be **multiple candidate nodes with the same cost** that we could expand – in BFS, DFS, and UCS the order in which these are expanded is arbitrarily determined, but maybe we can be **smart about it** and **use properties of the problem** to figure out **which** of these seemingly equivalent candidates **is more likely to get us to a solution**.

Consider the following grid representing a very simple path-finding problem:



In the above diagram intersections are circles, and streets are blue edges connecting neighbouring intersections. The start and goal nodes are shown. To make things simpler, for the time being, we assume all streets have the same **cost**, and let's make that cost equal to 1.

Inside each node, a number in **red** shows the cost of getting to that node from the start location. This is simply equal to the number of streets we have to traverse to get there. This also corresponds to a commonly used distance measure called the **Manhattan Distance** (the L1 norm) between two locations.

Next to each node, in **green**, is the order in which UCS expands the node to check if it's the goal. When expanding a node, its neighbours are added to the priority queue in clockwise order, starting at the top. *You should verify that the expansion order makes sense given how UCS works and the order in which the neighbours were added to the priority queue.* But you can see the pattern – all nodes at distance 1 were expanded before any neighbours at distance 2, and so on.

The end result is that the goal node was reached after most of the graph was explored. The problem here is that UCS employs no other information than distance from the start node. However, simply by looking at the graph, you can see that expanding nodes **to the right, or toward the bottom** of the map is taking us in **the wrong direction** even if they have a similar cost to nodes going toward the **left and top** where our goal node is.

Let's improve our search process so it will **explore more promising paths earlier** compared to other possible paths.

Heuristic Search, also called **A* (A-star) search** is a well known search algorithm that uses a **heuristic function** to **guess** which nodes in the search tree are likely to be closer to a goal state, the **heuristic** value is an **estimate** of the **cost for reaching a goal node** by going through an (as yet) **unexpanded node in the priority queue**.

Because it is a **guess**, it can not be used exclusively to guide search expansion order (if you think about it, in order to know exactly how far each node is from the goal, we'd have to have completed the search process on the entire tree!). Instead, **A*** search uses the heuristic function **in combination with the actual cost** of getting to a node from the initial state (this value is updated as nodes are expanded, just like with UCS).

The **heuristic cost** used by **A*** search to determine expansion order is:

$$f(n) = g(n) + h(n)$$

Where **f(n)** is the **heuristic cost** for some node **n**, and it consists of **g(n)** – the actual cost of reaching node **n** from the initial node, and **h(n)** a **guess of the cost to get to [the/a] goal state from n**.

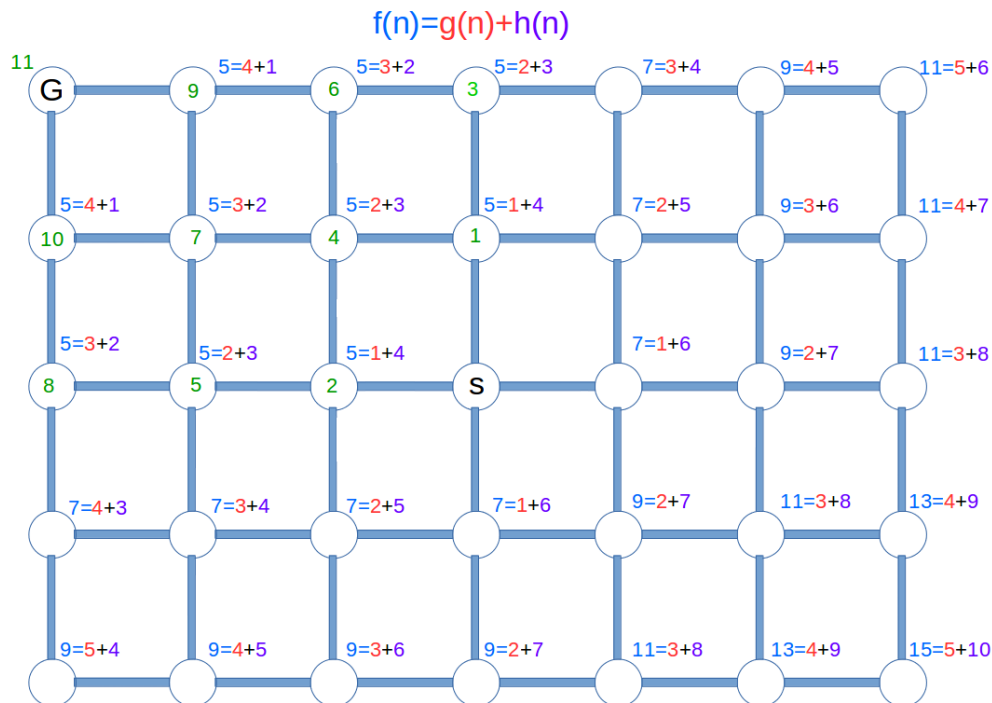
The **A*** search algorithm is **identical** to UCS, but we use the heuristic cost **f(n)** to order nodes in the priority queue. So if we have an implementation for UCS, modifying it to implement **A*** is straightforward.

In the example above, a **good guess** would be to use the **Manhattan distance between a node and the goal**. This happens to be a lucky guess for this problem because the Manhattan distance is the

actual distance between two nodes – it is very often **not** the case that we can find such an appropriate guess, so we'll talk about what makes a good guess below. For now, let's see how things change once we introduce the **heuristic cost** for each node into the search process.

The image below shows the same map, but this time we have annotated each node with the heuristic cost – you can see the two parts of it, $g(n)$ in red, and $h(n)$ in purple, as well as the total cost $f(n)$ in blue.

The expansion order is indicated inside the node, in green. You can immediately see that whereas UCS was exploring most of the map before reaching the goal, A* avoids expanding directions that move us farther away from the goal, and reaches the goal with significantly less exploration. In a large map, this could be a very significant reduction in the amount of search required to find the path to the goal!



Things to keep in mind:

- We don't actually know $g(n)$ in advance. We know $g(n)$ only for nodes that have been added to the priority queue, and it is not actually fixed (it could change!) until the moment when we expand the node (this is the same with UCS).

- For nodes with the same **heuristic cost**, expansion order is arbitrary – in the example above, neighbours of a node are added clockwise from top, and this is reflected in the expansion order above.

You should verify that the expansion order shown above is correct given the pseudocode for UCS/A* and the values of $f(n)$ shown in the figure.

What makes a good heuristic?

The interesting and often challenging part of implementing A* is finding a good heuristic to use in our search process. We can not just make an arbitrary guess – ***in order to preserve the optimality guarantee of UCS, the heuristic function has to satisfy certain conditions.***

Admissible Heuristic: A heuristic $h(n)$ is ***admissible*** if and only if

$$\forall n, 0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the ***true cost*** of getting to the goal node from node n – we do not know this cost, so proving that a heuristic is admissible is tricky! The Manhattan distance used above for the mapping example is admissible – convince yourself this must be the case!

Under the above conditions, $h(n)=0$ is admissible (i.e. using ***no*** heuristic function preserves the optimality of UCS – because it is UCS). Any non-zero admissible heuristic must have values that are ***less than, or at most equal*** to the true cost of getting from node n to the goal. Put in a different way, an ***admissible heuristic is optimistic*** – it does not over-estimate the cost of getting to the goal.

If the heuristic is NOT admissible then the A* process can return ***a sub-optimal path.***

Within the space of ***admissible heuristics*** for a given problem, the closer that $h(n)$ is to $h^*(n)$, the less search will be required to reach the ***goal state.***

It should also be noted that ***how you define $h(n)$ depends entirely on the problem you are trying to solve.*** The heuristic function is attached to the definition of the ***cost*** used to determine optimality for a path. As an example, let's say we change the cost for traversing a street in the maps example above. Instead of being constant, let's make the cost proportional to $1/S$ where S is the speed limit for the street. Faster streets have lower cost since we can travel through them in less time.

Under these conditions, the Manhattan distance is no longer appropriate (why?). Can you think of an admissible heuristic that we can use to carry out A* search when the cost of traversing a street is defined as inversely proportional to the maximum speed allowed down this street?

Defining a good heuristic function is the only actually ***interesting/intelligent*** part of solving a search problem of the type described above. The algorithms are all well known and you can find implementations for any language you care to use – but, defining a ***good, admissible heuristic*** is difficult, requires careful thought, and is what actually makes a particular ***search process*** better or worse than an ***off-the-shelf*** solution.

Exercise:

* Define a ***cost function*** for a ***realistic driving directions application*** which accounts for:

- Real-time traffic information (you must specify how this is encoded in the cost)
- Real-time updates for accidents and other disruptions that may close-off streets
- Intersections (cars do not move instantly from one street to the next, some intersections are worse than others)

* Given your cost function, suggest a **heuristic** that can be used to reduce the amount of search required to find a **good route**

Notice we stated we want a ‘good route’ since finding an **‘optimal route’** is no longer a well defined problem – for instance, using plan UCS at the start of the journey will find the **optimal path** at the start of the journey but this could change due to traffic and unexpected road events – therefore, we can **relax a bit** and think of a solid **heuristic** that allows us to find a **reasonably good solution** within an **acceptable amount of search**.

This allows us to **update the path** frequently – and even if we end up losing a bit on the optimality side, we gain a lot on the ability to respond to changes in the search space and the ability to run the search on less computationally powerful devices (such as smartphones).

In summary – it is **not always optimality** that matters. Develop your ability to decide what makes the most sense for your problem, and what will have the greatest usefulness for a user of your solution.

Crunchy Problems

Up to this point, we have focused on search problems with the specific property that **all the costs** are positive. The optimality guarantees of UCS and A* depend on this property. However, there may be situations where a specific problem may involve **negative weights** along some of the edges joining possible configurations – this make sense when a specific action involves a **reward** or **benefit** of some type for the agent.

As long as there are no negative weight cycles, an optimal solution is still possible. This is a fairly reasonable condition, and it is important to consider how we may go about solving such a search problem.

We can’t **get rid of the negative weights** – for instance, one may suggest adding a constant value to every edge in the graph so as to ensure all of them are positive. However, this changes the graph and our standard algorithms may no longer return the optimal path. **Why is that?** The optimal path may contain many edges – adding a constant to each of them can easily make the optimal path look longer than a sub-optimal one. So, we can’t take a shortcut, we have to find an algorithm that accounts for the negative weights in finding the optimal path.

The Bellman-Ford algorithms

The idea behind this method is to **iteratively** improve on the **shortest possible path** between an **initial node and all other nodes in the graph** until the shortest path to each is found – the goal is the

same as in Dijkstra's method, but because of the negative weights, the algorithm can not be **greedy** and instead **must consider all edges in the graph at each iteration**:

```
Initialize a shortest path array called 'dist' with one entry per node  
Set the shortest path cost to zero for the initial node  
Set the shortest path cost to 1e10 (or a suitably large number) for all other  
nodes  
Initialize a predecessor array called 'pred' with one entry per node  
set the predecessor value to -1 for all nodes (unknown at this point)  
  
Repeat N-1 times (for a graph with N nodes, M edges)  
  
For each edge (i,j), with cost w in the graph  
    if dist[i]+w < dist[j] // Found a shorter path to j  
        dist[j]=dist[i]+w  
        pred[j]=i
```

The algorithm itself is straightforward, however, it has a complexity of

$O(N * M)$

Which is significantly worse than the **$O(N+M)$** of naive UCS, or the **$O(M \log N)$** of Dijkstra's method implemented with regular binary heaps for the priority queue. On a large graph, this becomes prohibitively expensive.

Which brings us back to the comment regarding **heuristic functions** above: **Often, we prefer a 'good enough' solution that can be obtained in a reasonable amount of time, over an 'optimal' solution that can not.**